

02 — CLI **oc**: Installazione e Configurazione

Modulo: 2 — Navigazione e Strumenti

Versione: OpenShift 4.14+ su Azure ARO

Obiettivi di Apprendimento

- Scaricare e installare la CLI **oc** su Linux, macOS e Windows
 - Verificare la compatibilità della versione **oc** con il cluster OCP
 - Aggiungere **oc** al PATH di sistema in modo permanente
 - Comprendere la relazione tra **oc** e **kubectl** e usarli in modo intercambiabile
-

1. Cos'è la CLI **oc**

oc è la **command-line interface ufficiale di OpenShift**. È un superset di **kubectl**: include tutti i comandi Kubernetes standard e aggiunge comandi specifici per le funzionalità OCP (Route, BuildConfig, DeploymentConfig, Project, SCC, ecc.).

```
oc = kubectl + comandi specifici OpenShift
```

Nota: Tutti i comandi **kubectl** funzionano identicamente con **oc**. Se conosci già **kubectl**, puoi usare **oc** subito senza imparare nuovi comandi base.

2. Download della CLI **oc**

2.1 Dalla Console Web (metodo consigliato)

La console OCP fornisce sempre il binario **oc** compatibile con la versione esatta del cluster:

```
Console Web → ? (icona Help, in alto a destra) →  
"Command line tools" →  
Scaricare il binario per il proprio OS
```

2.2 Da Red Hat Mirror

```
# Verificare la versione del cluster prima del download  
# (da eseguire se si ha già accesso al cluster)  
oc version --short 2>/dev/null || \  
curl -s https://api.<cluster>:6443/version | python3 -m json.tool
```

URL di download ufficiale:

- **Red Hat Downloads:** <https://console.redhat.com/openshift/downloads>
- **Mirror Red Hat:** <https://mirror.openshift.com/pub/openshift-v4/clients/ocp/>

2.3 Su Azure ARO — URL specifico

```
# Ottenere l'URL API del cluster ARO
API_URL=$(az aro show \
  --name myAROCluster \
  --resource-group myResourceGroup \
  --query "apiserverProfile.url" \
  --output tsv)
echo "API URL: $API_URL"

# Download diretto dalla console ARO
CONSOLE_URL=$(az aro show \
  --name myAROCluster \
  --resource-group myResourceGroup \
  --query "consoleProfile.url" \
  --output tsv)
echo "Scaricare oc da: ${CONSOLE_URL}/command-line-tools"
```

3. Installazione per Sistema Operativo

3.1 Linux (RHEL / Fedora / Ubuntu)

```
# Scaricare il pacchetto tar.gz (esempio versione 4.14)
curl -LO https://mirror.openshift.com/pub/openshift-
v4/clients/ocp/4.14.0/openshift-client-linux.tar.gz

# Estrarre i binari oc e kubectl
tar -xzf openshift-client-linux.tar.gz

# Spostare i binari nel PATH di sistema
sudo mv oc kubectl /usr/local/bin/

# Impostare i permessi di esecuzione
sudo chmod +x /usr/local/bin/oc /usr/local/bin/kubectl

# Verifica installazione
oc version --client
# Output atteso:
# Client Version: 4.14.x
# Kustomize Version: v5.0.x
```

Aggiunta permanente al PATH (se installato in directory custom):

```
# Aggiungere al file ~/.bashrc o ~/.bash_profile
echo 'export PATH=$PATH:/path/to/oc/directory' >> ~/.bashrc
source ~/.bashrc
```

Completamento automatico dei comandi (bash):

```
# Abilitare il completamento per la sessione corrente
source <(oc completion bash)

# Abilitare in modo permanente
oc completion bash | sudo tee /etc/bash_completion.d/oc > /dev/null
```

3.2 macOS

```
# Con Homebrew (metodo consigliato)
brew install openshift-cli

# Oppure manualmente
curl -LO https://mirror.openshift.com/pub/openshift-
v4/clients/ocp/4.14.0/openshift-client-mac.tar.gz
tar -xzf openshift-client-mac.tar.gz
sudo mv oc /usr/local/bin/
sudo chmod +x /usr/local/bin/oc

# Verifica
oc version --client
```

⚠ Attenzione macOS: Al primo avvio, macOS potrebbe bloccare `oc` come "applicazione non verificata". Per sbloccarla:

System Settings → Privacy & Security → Allow Anyway

Oppure da terminale: `xattr -d com.apple.quarantine /usr/local/bin/oc`

Completamento automatico (zsh — default su macOS):

```
# Aggiungere al file ~/.zshrc
echo 'source <(oc completion zsh)' >> ~/.zshrc
source ~/.zshrc
```

3.3 Windows

Con winget (Windows Package Manager):

```
# Cercare il pacchetto
winget search openshift

# Installare
winget install RedHat.openshift-client
```

Installazione manuale:

```
# Scaricare il file zip
Invoke-WebRequest `
  -Uri "https://mirror.openshift.com/pub/openshift-
v4/clients/ocp/4.14.0/openshift-client-windows.zip" `
  -OutFile "openshift-client-windows.zip"

# Estrarre
Expand-Archive openshift-client-windows.zip -DestinationPath C:\OpenShift\

# Aggiungere al PATH utente (permanente)
[Environment]::SetEnvironmentVariable(
  "PATH",
  "$([Environment]::GetEnvironmentVariable('PATH', 'User'));C:\OpenShift",
  "User"
)

# Aprire un nuovo terminale e verificare
oc version --client
```

 **Attenzione Windows:** Dopo aver modificato il PATH, è necessario **chiudere e riaprire PowerShell** (o il terminale) per applicare le modifiche.

4. Verifica dell'Installazione e Compatibilità

```
# Verificare la versione del client
oc version --client
# Output atteso:
# Client Version: 4.14.x
# Kustomize Version: v5.0.x

# Dopo il login al cluster, verificare client e server
oc version
# Output atteso:
# Client Version: 4.14.x
# Kustomize Version: v5.0.x
# Server Version: 4.14.x
# Kubernetes Version: v1.27.x
```

Regola di Compatibilità delle Versioni

La CLI **oc** è **retrocompatibile e forward-compatibile** di ± 1 versione minore rispetto al server:

Versione Server	Versione Client	Compatibile?
4.14.x	4.14.x	✓ Consigliato
4.14.x	4.13.x	✓ Supportato
4.14.x	4.15.x	✓ Supportato
4.14.x	4.12.x	⚠ Non consigliato
4.14.x	4.10.x	✗ Non supportato

⚠ **Attenzione:** Usare sempre una versione client allineata a quella del cluster per evitare comportamenti imprevisti con le nuove API.

5. **oc** vs **kubectl** — Comandi Equivalenti e Specifici

Comandi equivalenti (identici)

```
# Questi comandi funzionano identicamente con oc e kubectl
oc get pods           # = kubectl get pods
oc get nodes          # = kubectl get nodes
oc apply -f manifest.yaml # = kubectl apply -f manifest.yaml
oc delete pod my-pod   # = kubectl delete pod my-pod
oc describe deployment myapp # = kubectl describe deployment myapp
oc logs my-pod -c container # = kubectl logs my-pod -c container
oc exec -it my-pod -- bash # = kubectl exec -it my-pod -- bash
oc port-forward svc/myapp 8080 # = kubectl port-forward svc/myapp 8080
```

Comandi specifici **oc** (non disponibili in **kubectl**)

```
# Gestione Project (namespace con policy aggiuntive OCP)
oc new-project my-project
oc projects
oc project my-project # cambia progetto corrente

# Login e autenticazione
oc login https://api.cluster:6443

# Deploy rapido da immagine o sorgente
oc new-app nodejs~https://github.com/myorg/app.git
oc new-app --image=registry.access.redhat.com/ubi9/httpd-24

# Esposizione servizio come Route
oc expose svc/my-service
```

```
# Shell interattiva in un Pod
oc rsh my-pod-name

# Debug (avvia un Pod clone per troubleshooting)
oc debug pod/my-pod
oc debug node/worker-0.example.com

# Gestione rollout DeploymentConfig (legacy OCP)
oc rollout latest dc/my-app
oc rollback dc/my-app

# Gestione build
oc start-build my-buildconfig
oc cancel-build my-build-1
oc logs -f bc/my-buildconfig

# Amministrazione cluster
oc adm top nodes
oc adm top pods
oc adm policy add-cluster-role-to-user cluster-admin myuser
oc adm must-gather # raccolta diagnostica
```

Tabella Riassuntiva

Operazione	kubectl	oc
Gestione risorse standard	✓	✓
Login al cluster	✗	✓ oc login
Gestione Project	✗	✓ oc new-project
Shell in Pod	exec -it -- sh	✓ oc rsh
Debug avanzato	✗	✓ oc debug
Deploy da sorgente (S2I)	✗	✓ oc new-app
Route (esposizione HTTP)	✗	✓ oc expose
Gestione Build	✗	✓ oc start-build
Comandi adm (admin)	✗	✓ oc adm

6. Esercizi Pratici

Esercizio 1 — Installazione e Verifica

Obiettivo: installare **oc** e verificarne il funzionamento.

1. Scaricare **oc** per il proprio sistema operativo dalla console web ARO o da Red Hat Mirror
2. Spostare il binario in una directory nel PATH
3. Aprire un nuovo terminale e verificare:

```
oc version --client  
which oc
```

4. Abilitare il completamento automatico per la propria shell
5. Digitare `oc <TAB><TAB>` e verificare la lista dei comandi disponibili

Soluzione attesa:

```
Client Version: 4.14.x  
/usr/local/bin/oc      (Linux/macOS)  
C:\OpenShift\oc.exe    (Windows)
```

Esercizio 2 — Esplorazione Comandi `oc`

Obiettivo: familiarizzare con i comandi specifici di `oc`.

1. Dopo aver eseguito il login al cluster (vedi modulo [03-autenticazione-contesti.md](#)):

```
# Visualizzare tutti i progetti disponibili  
oc projects  
  
# Visualizzare le informazioni sul cluster  
oc cluster-info  
  
# Visualizzare i tipi di risorse disponibili nell'API del cluster  
oc api-resources | head -30  
  
# Verificare la differenza tra oc e kubectl  
oc get nodes -o wide  
kubectl get nodes -o wide    # stesso output
```

2. Trovare il comando `oc` equivalente di `kubectl apply --dry-run=client`:

```
oc apply -f manifest.yaml --dry-run=client  
# → identico! oc accetta tutti i flag di kubectl
```

Soluzione commentata: `oc` e `kubectl` sono intercambiabili per tutte le operazioni standard Kubernetes; `oc` aggiunge i comandi specifici OCP come `new-project`, `rsh`, `debug`, `expose`.

7. Riepilogo dei Punti Chiave

- **oc** si scarica dalla **Console Web del cluster** oppure da mirror.openshift.com — usare sempre la versione allineata al cluster (± 1 versione minore)
 - Su **Linux**: spostare il binario in `/usr/local/bin/`; su **Windows**: aggiungere la directory al PATH utente
 - Abilitare sempre il **completamento automatico** (`oc completion bash/zsh`) per aumentare la produttività
 - **oc** è un **superset di kubectl**: tutti i comandi `kubectl` funzionano identicamente con `oc`
 - I comandi esclusivi di `oc`: `login`, `new-project`, `rsh`, `debug`, `new-app`, `expose`, `start-build`, `adm`
-

Riferimenti

- [Installing oc CLI - Red Hat](#)
- [OpenShift CLI Reference](#)
- [oc vs kubectl - Red Hat Blog](#)
- [OpenShift Mirror Downloads](#)